

POLITECNICO
MILANO 1863

Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

Thread & Concorrenza

My Contacts:
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

Thread

- La classe Thread implementa un’interfaccia chiamata Runnable
- L’interfaccia Runnable espone come unico metodo, **run()**
- Questo consente di avere un metodo alternativo per creare un thread

Esempio:

```
public class HelloThread extends Thread {
@Override
public void run() {
System.out.println("Hello from a thread!");
}
public static void main(String args[])
{
(new HelloThread()).start();
}
}
```

Come creare un Thread customizzato

1. Definire una classe che implementi Runnable. Questa classe può estenderne altre, ed ha tutte le caratteristiche di una classe java.
2. Creare un'istanza della classe
3. Creare un oggetto Thread e passare come input l'oggetto che implementa Runnable
4. Attiva il metodo «run()» attraverso il metodo «start()» dell'oggetto Thread creato

Nota: All'oggetto thread possiamo anche passare una classe anonima che implementi il metodo run()…o una lambda expression.

Esempio

```
public class HelloRunnable implements Runnable
{
public void run()
{
System.out.println("Hello from a thread!");
}

public static void main(String args[])
{
(new Thread(new HelloRunnable())).start();
}
}
```

Attenzione!

- Quando avete risorse condivise tra più thread, occorre accedervi con cautela. Per modificare una risorsa, occorre accedervi in mutual esclusione.
- Vogliamo che alcune sequenze di istruzioni vengano seguite in maniera “isolate”, senza “interleaving” di operazioni da diversi thread. Parliamo in questo caso di **sequenze atomiche.**

Accesso ad Oggetti Condivisi

- Synchronized:
  - Implicit
  - Explicit
  - Static

Esempio Synchronized Implicit

```
public synchronized int method1()
{
//body
}

public synchronized int method2()
{
//body
}
```

Esempio Synchronized Explicit

```
final private Object lock1 = new Object();
 final private Object lock2 = new Object();
public void method() {
synchronized(lock1) {
//your code}
 }
public void addB() {
synchronized(lock1) {
//other code
}}

public void addB() {
synchronized(lock2) {
//other code
}}
```

Liveness propery

- Una applicazione deve essere eseguita in un tempo accettabile.
  Possibili problemi:
  - Deadlock
  - Starvation
  - Livelock

Deadlock

- Due o più Thread sono bloccati per sempre, aspettandosi l'un l'altro

Thread1	**Thread2**
LockRequest (A)	LockRequest(B)
LockRequest (B)	LockRequest(A)
Release (A)	Release(B)

Il primo thread ha il lock di A, ma lo lascia solo dopo aver ottenuto il lock di B. Il secondo thread ha il lock di B, ma lo lascia solo dopo aver ottenuto il lock di A.

Starvation

- «Starvation» è la situazione in cui un thread ha difficoltà ad accedere ad una certa risorsa condivisa, e quindi fa fatica a procedere con la sua esecuzione.

Example:

Se un Thread accede ad una risorsa e la occupa per un lasso di tempo molto lungo, tutti gli altri thread che necessitano di quella risorsa saranno bloccati.

Livelock

- E’ di fatto un errore di design che porta ad una sequenza ciclica di operazioni per gestire il proseguire dell’esecuzione

Example:

Due thread che inziano una sequenza infinita di «Prego, vai pure». Data una risorsa A, Il Thread 1 darà precedenza al Thread 2 mentre il Thread 2 da precedenza al Thread 1

Guarded Blocks

- Per evitare questa condizione, è possibile mettere i thread «in pausa» fino ad una determinata condizione, per poi svegliarli.

  Come?

  - wait()
  - notify () // non determinismo
  - notifyAll()

Esercizio: Contatore

Si progetti una classe Java Contatore che permetta l'accesso concorrente ad un contatore che:

- Inizialmente vale 0 e puo' arrivare a 10;

- Quando il contatore arriva a 10, il metodo reset() consente di tornare a 0. L'operazione di reset deve essere possibile solo se il contatore vale 10.

I metodi **incrementa()** e **decrementa()** consentono l'incremento e il decremento di una unità per volta.

- Le operazioni di incremento e decremento devono sempre mantenere il valore del contatore tra 0 e 10.

Esercizio: Contatore

Iniziamo con l’implementare una classe ContoInBanca, che ha come suo unico attributo un intero: valore

Esercizio: Contatore

Iniziamo con l'implementare una classe ContoInBanca, che ha come suo unico attributo un intero: valore

Creiamo poi un thread publisher ed uno subscriber. Entrambe queste classi estendono Thread e, all'interno del metodo «run()» chiamano rispettivamente la funzione incrementa e decrementa

Esercizio: Contatore

Iniziamo con l'implementare una classe ContoInBanca, che ha come suo unico attributo un intero: valore

Creiamo poi un thread publisher ed uno subscriber. Entrambe queste classi estendono Thread e, all'interno del metodo «run()» chiamano rispettivamente la funzione incrementa e decrementa.

Cosa succede se non sincronizzo in alcun modo il contatore?

Esercizio: Contatore

Iniziamo con l'implementare una classe ContoInBanca, che ha come suo unico attributo un intero: valore

Creiamo poi un thread publisher ed uno subscriber. Entrambe queste classi estendono Thread e, all'interno del metodo «run()» chiamano rispettivamente la funzione incrementa e decrementa.

Cosa succede se non sincronizzo in alcun modo il contatore?

Viene lanciata un'eccezione in quanto il programma non rispetta i requisiti!!

Per vedere il comportamento senza sincronizzazione, vedere il codice PublishSubscribeNoSync

Esercizio: Contatore

Occorre quindi implementare la logica di sincronizzazione: dove la implemento?

1. Il primo modo**, sebbene non sia il più corretto,** consiste nell'implementare le operazioni che posso fare sul conto in banca e la sincronizzazione, all'interno dei due Thread, publish e subscribe.

Per vedere il comportamento senza sincronizzazione, vedere il codice PublishSubscribeNoSync

Esercizio: Contatore

Occorre quindi implementare la logica di sincronizzazione: dove e come la implemento?

1. Sincronizzare i metodi, ed usare nella run dei metodi sincronizzati.

2. Sincronizzare parte dei metodi sincronizzati.

3. Sincronizzare parte dei metodi con un blocco synchronized su un oggetto.

4. Sincronizzare parte dei metodi con un blocco synchronized su un oggetto ausiliario.

Per vedere il comportamento senza sincronizzazione, vedere il codice PublishSubscribeNoSync

Esercizio: Contatore

Hint: Fate sempre attenzione a quando **rilasciare** un lock. Se acquisite un lock in un blocco sincronizzato mentre siete in un while true senza uscita…non lo rilascerete mai!

Per vedere il comportamento senza sincronizzazione, vedere il codice PublishSubscribeNoSync

Esercizio: Guarded Counter

Si progetti la classe Java Contatore con guarded blocks: se l'operazione del thread di incrementare/decrementare rischia di corrompere il contatore (superare il massimo od il minimo) il thread viene messo in pausa.

Esercizio: Stack

- Si consideri la classe Stack che implementa la seguente interfaccia:

```
1 interface Stack_interface {
2 void push(int n);
3 int pop();//estrae e ritorna l'elemento in cima allo stack
4 boolean is_empty();
5 }
```

Esercizio: Synchronized Stack

Si ipotizzi che oggetti della classe Stack possano essere manipolati concorrentemente e che quindi si debba garantire un accesso corretto. Se un task cerca di effettuare l'operazione pop quando l'oggetto stack è vuoto, esso dovra' essere sospeso fino a che un altro task non avra' effettuato un'operazione push (a meno che non intervenga una InterruptedException).

Esercizio: Synchronized Map

Si consideri una classe Map personalizzata contenente dati. Ogni riga della tabella contiene una chiave intera ed una stringa come valore. La tabella offre le seguenti operazioni:

Esercizio: Synchronized Map

Si consideri una classe Map personalizzata contenente dati. Ogni riga della tabella contiene una chiave intera ed una stringa come valore. La tabella offre le seguenti operazioni:

- searchAndGet(key) riceve come parametro una chiave e restituisce tutte le stringhe memorizzate nella tabella di dati con quella chiave, cancellandola dalla tabella stessa.

Esercizio: Synchronized Map

Si consideri una classe Map personalizzata contenente dati. Ogni riga della tabella contiene una chiave intera ed una stringa come valore. La tabella offre le seguenti operazioni:

- searchAndGet(key) riceve come parametro una chiave e restituisce tutte le stringhe memorizzate nella tabella di dati con quella chiave, cancellandola dalla tabella stessa.

- Insert () riceve un dato (stringa), con chiave associata, e lo memorizza.

Esercizio: Synchronized Map

Si consideri una classe Map personalizzata contenente dati. Ogni riga della tabella contiene una chiave intera ed una stringa come valore. La tabella offre le seguenti operazioni:

- searchAndGet(key) riceve come parametro una chiave e restituisce tutte le stringhe memorizzate nella tabella di dati con quella chiave, cancellandola dalla tabella stessa.

- Insert () riceve un dato (stringa), con chiave associata, e lo memorizza.

La tabella viene usata in un ambiente concorrente da più thread. Poiche' essa ha una dimensione predefinita, quando risulta piena, l'invocazione dell'operazione *insert* sospende il task che invoca l'operazione in attesa che venga liberato spazio dall'invocazione di *searchAndGet*.

Si fornisca un'implementazione Java della classe Tabella.

Esercizio: Sala da Ballo

Si consideri una classe che gestisce una sala da ballo. In questa sala ci sono Dame e Cavalieri che ballano insieme. In particolare:

Esercizio: Sala da Ballo

Dama: nella sala da ballo ci sono dame disponibili a ballare e dame tristi. Una dama triste e' una dama che e' appena stata scaricata dal cavaliere, e non ha nessuno con cui parlare.

Non appena ci sono due dame tristi, queste diventano disponibili a ballare.

Esercizio: Sala da Ballo

Cavaliere: un cavaliere cicla tra gli stati: in coppia, in crisi e senza dama.
- Quando e’ in coppia, aspetta un tempo random e poi va in crisi.
- Quando e’ in crisi scarica la dama.
- Quando e’ da solo, richiede una dama alla sala da ballo (se non ce ne sono, aspetta). Se non ci sono dame aspetta che una dama si liberi.

- Se la dama scaricata andrebbe tra quelle in crisi, aspetta che abbia un’altra dama a farle compagnia.

Esercizio: Sala da Ballo

Cavaliere: un cavaliere cicla tra gli stati: in coppia, in crisi e senza dama.
- Quando e’ in coppia, aspetta un tempo random e poi va in crisi.
- Quando e’ in crisi scarica la dama.
- Quando e’ da solo, richiede una dama alla sala da ballo (se non ce ne sono, aspetta). Se non ci sono dame aspetta che una dama si liberi.

- Se la dama scaricata andrebbe tra quelle in crisi, aspetta che abbia un’altra dama a farle compagnia.

Esercizio: Sala da Ballo

Sala da Ballo: gestisce le dame della sala da ballo.

Si fornisca un’implementazione Java della sala da ballo, con dame e cavalieri.

Gestione Avanzata dei Lock

I meccanismi di lock visti sono basici, ma potremmo avere una gestione dei lock più granulare.

- lock
- unlock
- tryLock
- isLocked